AI-Generated Website Creation using Drupal Canvas: Candidate Submission

Oct 9, 2026 · @Purvita

Summary and assumptions

I propose a hybrid pattern: the AI engine generates HTML outside Drupal, a mapping step converts it into approved components, and Drupal Canvas is where authors review, edit and publish it. The prototype shows this working on a local Drupal CMS 2 site.

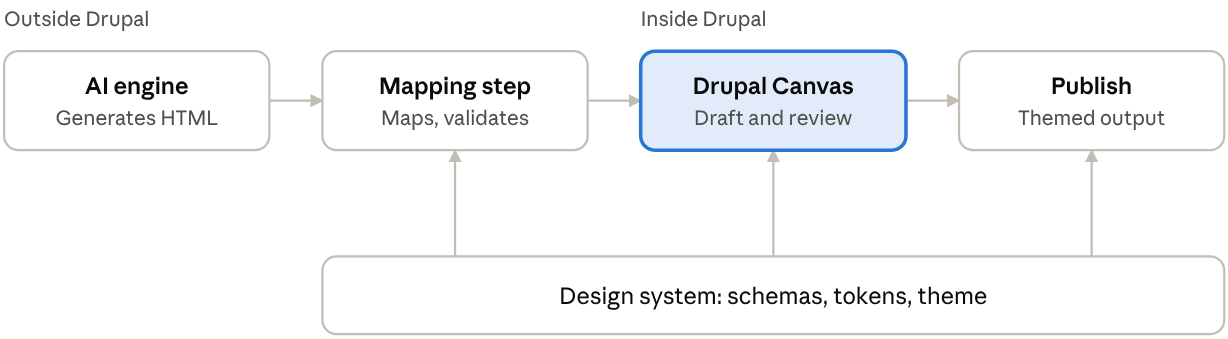
Assumptions

- The company's AI Content Engine, Figma design system and production theme were not available. The generated HTML page, my custom theme and my component set stand in for them.
- The mapping from HTML to components was carried out once, with AI-assisted tooling that I directed and reviewed. It is not yet an automated pipeline. Section 5.3 describes how that step would be automated and validated.
- Page content is sample text for demonstrating structure. It is not medically reviewed.

5.1 Architecture proposal

The pattern is hybrid: the AI engine generates HTML outside Drupal, while composition, governance and rendering stay in Drupal Canvas and the native theme. This keeps the engine independent and every published page inside the governed stack.

Generation stays outside Drupal; publishing passes through approved components



short architecture · 4 steps, 1 design-system layer

Sections that match no component never reach the page as raw HTML; they are flagged on the draft for the author to resolve.

- **Generation.** The AI Content Engine produces the HTML and handles compliance outside Drupal.
- **Mapping.** A step at the Drupal boundary maps each section to an approved component, validates it against the component schema, and outputs a component tree as JSON.
- **Canvas.** Canvas stores the page as a tree of components with typed props. Authors review and edit it visually, and the theme renders it.
- **Publishing.** Pages arrive as drafts and are published after editorial approval, for example with Content Moderation.

5.2 Working prototype

The prototype turns the AI-generated page `sample-input.html` into a Canvas page built only from design-system components. It runs locally on Drupal CMS 2 with Drupal Canvas (Drupal 11, DDEV), and setup steps are in the README.

Output. A Canvas page on the local site, rendered by the Drupal theme. Header and footer sit in the Canvas Page template, outside the main content area, as in the source HTML. The body contains hero, benefits, FAQ, resources and CTA in the same order as the input.

Components built. Header, hero, section heading, benefit card, FAQ, resource card, CTA and footer. Each is a Single Directory Component with a props schema, a Twig template and its own CSS, styled only through the theme's design tokens.

5.3 Content transformation strategy

The transformation turns each section of the HTML into one approved component whose props hold only text, links and image references. Styling and layout never travel with the content, because the component and theme own them.

Component identification. The mapper reads the semantic structure first (header, main sections, footer, details/summary, lists), then section classes and heading text as secondary hints. Each section is assigned to exactly one component type.

Mapping logic.

HTML element	Component	Props
`header.site-header`	header	brand, nav links, button label and URL
`section.hero`	hero	heading, lead text, primary button, secondary link, image URL and alt text
`#benefits` heading and intro	section heading	heading, intro text
`div.benefit`	card	icon, title, text
`section.faq` with `details`	FAQ	heading, items (question, answer)
`section.resources` list items	resource card	title, text, link label, link URL
`section.cta`	CTA	heading, text, button label and URL
`footer.site-footer`	footer	brand, links, disclaimer lines

Validation and error handling. Each mapped component is checked against its props schema: required fields, types, URLs, alt text and heading order. Inline styles, scripts and unknown tags are dropped. A section that matches no component is flagged on the draft for an author to resolve, never published as raw HTML, and a missing required field keeps the page in draft.

Sample mapping output. The tree below is an illustrative extract of the component structure. The full file exported from the built page, `output-components.json`, is submitted with this document.

```
{
  "page": { "title": "Galvus Patient Support", "status": "draft" },
  "template": ["header", "footer"],
  "content": [
    { "component": "hero", "props": {
        "heading": "Take a confident step in managing your type 2 diabetes",
        "primary_cta": { "label": "Explore support resources", "url":
"#support" } } },
    { "component": "faq", "props": {
        "heading": "Frequently asked questions",
        "items": [ { "question": "What is Galvus used for?", "answer": "..."
} ] } },
    { "component": "cta", "props": {
        "heading": "Ready to get started?",
        "button": { "label": "Join the programme", "url": "#" } } }
  ]
}
```

5.4 Design system integration

The design system is enforced by construction: pages can only be assembled from registered components, and those components can only be styled through the theme's tokens.

How the Drupal theme is used. The custom theme owns all visual decisions. It defines design tokens for colours, typography, spacing and radii, and every component's CSS references those tokens only. In this prototype the tokens were taken from the generated page's CSS variables, standing in for tokens exported from the company's Figma design system.

How approved components are enforced.

- Only components in the Canvas library can be placed, and their props schemas leave no free-form HTML or inline style field.
- Header and footer live in the Canvas Page template, so authors cannot remove or alter them per page.
- Figma layout rules, such as which components may appear in which layout, would become allowed-component constraints on template regions.

How accessibility is maintained.

- Components output semantic HTML: landmarks, ordered headings, and native details/summary for a keyboard-friendly FAQ.
- Image components require alt text, and the theme provides a skip link, visible focus outlines and sufficient contrast, so one fix improves every page.
- Planned checks before release: automated testing with axe and Lighthouse, plus a manual keyboard and screen-reader pass.

How responsive layout and brand consistency are preserved. Breakpoints and grid behaviour live once in each component's CSS, so pages adapt like the source: four benefit columns on desktop, two on tablet, one on mobile. Fonts, colours and spacing come from shared tokens, so a brand update changes every page.

5.5 Trade-off analysis

I chose hybrid because it gives the speed of AI generation without giving up Drupal's governance, at the cost of building and maintaining one mapping step.

Pattern	Strength	Weakness
Fully coupled (AI inside Drupal)	One system to run	Ties the AI engine to Drupal's release cycle and limits reuse elsewhere
Headless (AI page served as is)	Fastest to ship	Bypasses the theme, design system and review workflow
Hybrid (chosen)	Engine stays independent, output is governed	Needs a maintained mapping and validation layer

Scalability. Adding a component means adding one schema, one template and one mapping rule, so the library grows without changing the pipeline. Rules can be configuration-driven, and the same mapper can serve many campaigns, languages and brands that share a design system.

Governance. Every AI page enters as a draft, is checked by schema validation, and passes human review before publication. Authors work only with approved components, and the mapping log gives an audit trail of what the engine produced and what was changed.

Limitations.

- The mapping was performed once with AI-assisted tooling and is not yet an automated, tested converter.
- Rule-based matching is fragile for unusual or deeply nested layouts.
- Only a small component set was built, and the Figma system was approximated with tokens from the generated page.
- Accessibility is designed in but not yet verified with automated and assistive-technology testing, and Canvas is new, so its APIs may change.

Future enhancements.

- Build the mapper as a Drush command or service that writes Canvas pages through Drupal APIs, with tests on the mapping rules.
- Use an LLM only to suggest mappings for unmatched sections, always schema-validated and author-reviewed.
- Add automated accessibility checks and visual regression tests to the publishing workflow.
- Sync design tokens from Figma, and add layout rules that restrict which components appear in which regions.

Submission contents

The package contains the following items, matching the expected submission in the brief.

Item	File or link
Architecture write-up	This document
Sample input HTML	`sample-input.html`
Component-mapped output	`component-mapping.md` and `output-components.json`
Prototype source code and setup instructions	Project folder (theme, components) with `README.md`
Demo walkthrough	Short screen recording of the Canvas editor and the published page
Assumptions	Summary section above and `assumptions.md`